20 задач для Keyestudio EASY-Plug Starter Kit

От простого к сложному | Урок 1.3+

Уровень 1: Первые шаги (LED + основы)

Задача 1. Маяк

Модули: 1 светодиод (Red LED) **Суть:** Напиши программу, которая мигает красным светодиодом: 2 секунды горит, 1 секунда не горит. **Цель:** Освоить `digitalWrite`, `delay`, понять связь между числом в `delay()` и реальным временем.

```cpp
// Подсказка: delay(2000) = 2 секунды
```

Задача 2. SOS

Модули: 1 светодиод **Суть:** Запрограммируй сигнал SOS азбукой Морзе: три короткие вспышки (100 мс), три длинные (500 мс), три короткие. Пауза 2 секунды — и повтор. **Цель:** Использовать цикл `for` для повторяющихся блоков.

Задача 3. Светофор

Модули: 3 светодиода (красный, жёлтый, зелёный) **Суть:** Воспроизведи работу светофора: зелёный 3 с → жёлтый мигает 5 раз → красный 3 с → цикл. **Цель:** Управлять несколькими выходами, не забывать выключать предыдущий.

Задача 4. Бегущий огонь

Модули: 3 светодиода **Суть:** Свет «перебегает» от одного светодиода к другому: красный → жёлтый → зелёный → жёлтый → красный, каждый горит 300 мс. **Цель:** Понять последовательность команд и ритм переключений.

Задача 5. Диммер на коде

Модули: 1 светодиод (подключён к PWM-пину) **Суть:** Плавно увеличивай яркость светодиода от 0 до 255, затем уменьшай обратно. Используй `analogWrite()` и цикл `for`. **Цель:** Познакомиться с ШИМ (PWM) — аналоговым управлением через цифровой выход.

```cpp
// Подсказка: analogWrite(pin, 0..255);
```

Уровень 2: Датчики — робот учится «чувствовать»

Задача 6. Тревога: датчик газа

Модули: датчик газа (MQ-2/MQ-5), 1 светодиод **Суть:** Считывай значение с датчика газа через `analogRead()`. Если значение выше порога (например, 400) — включи светодиод. Ниже — выключи. **Цель:** Освоить `analogRead()`, условие `if/else`, понять разницу между цифровым и аналоговым сигналом.

```cpp
int gasValue = analogRead(GAS_PIN);
if (gasValue > 400) { ... }
```

Задача 7. Охранник: ИК-датчик

Модули: ИК-датчик препятствий, 1 светодиод **Суть:** Когда ИК-датчик обнаруживает объект перед собой — зажги светодиод. Убрали объект — погас. **Цель:** Работа с `digitalRead()`, понимание логических уровней HIGH/LOW от датчика.

Задача 8. Линейка из звука: ультразвуковой датчик

Модули: УЗ-датчик расстояния (HC-SR04), Serial Monitor **Суть:** Измерь расстояние до объекта и выведи его в сантиметрах в Serial Monitor каждые 500 мс. **Цель:** Освоить работу с `pulseIn()`, формулу расчёта расстояния, вывод данных в `Serial.println()`.

```cpp
// Формула: distance_cm = duration / 58;
```

Задача 9. Парктроник

Модули: УЗ-датчик, 3 светодиода (зелёный, жёлтый, красный) **Суть:** Зелёный горит, если расстояние > 30 см. Жёлтый — от 10 до 30 см. Красный — менее 10 см. **Цель:** Комбинировать датчик и несколько выходов через `if / else if / else`.

Задача 10. Термометр в мониторе

Модули: датчик температуры (LM35 или аналог), Serial Monitor **Суть:** Считай аналоговое значение с датчика температуры, переведи в градусы Цельсия и выведи в Serial Monitor. **Цель:** Математические операции с данными датчиков, калибровка.

```cpp
float tempC = analogRead(TEMP_PIN) * 5.0 / 1024.0 * 100.0;
```

Уровень 3: Управляем механизмами

Задача 11. Сервопривод — стрелка прибора

Модули: серво-мотор **Суть:** Поверни серво на 0°, подожди 1 с, на 90°, подожди 1 с, на 180°, подожди 1 с — и обратно. **Цель:** Подключить библиотеку `Servo.h`, использовать `servo.write()`.

```cpp
#include <Servo.h>
Servo myServo;
myServo.attach(pin);
myServo.write(90); // поворот на 90°
```

Задача 12. Плавный поворот

Модули: серво-мотор **Суть:** Плавно поворачивай серво от 0° до 180° с шагом 1° и задержкой 15 мс, затем обратно. **Цель:** Использовать цикл `for` с серво для создания плавного движения.

Задача 13. Радар: серво + УЗ

Модули: серво-мотор, УЗ-датчик, Serial Monitor **Суть:** Серво поворачивается от 0° до 180° с шагом 10°. На каждом шаге измерь расстояние и выведи в Serial: `"Угол: 30° — Расстояние: 25 см"`. **Цель:** Синхронизация двух устройств, форматированный вывод.

Уровень 4: Комплексные системы

Задача 14. Умный свет

Модули: датчик освещённости (фоторезистор), 1 светодиод **Суть:** Чем темнее в комнате — тем ярче горит светодиод (через `analogWrite`). При полном свете — светодиод выключен. **Цель:** Функция `map()` для пересчёта диапазонов, обратная зависимость.

```cpp
int brightness = map(lightValue, 0, 1023, 255, 0);
analogWrite(LED_PIN, brightness);
```

Задача 15. Часы в мониторе

Модули: модуль реального времени (RTC DS1302/DS1307), Serial Monitor **Суть:** Считай текущее время с модуля RTC и выводи в Serial Monitor в формате `ЧЧ:ММ:СС` каждую секунду. **Цель:** Работа с библиотекой RTC, чтение данных по протоколу.

Задача 16. Будильник

Модули: модуль времени (RTC), пассивный зуммер, 1 светодиод **Суть:** Задай время будильника в коде. Когда текущее время совпадает — включи зуммер и мигающий светодиод на 10 секунд. **Цель:** Сравнение значений времени, комбинация вывода звука и света.

Задача 17. Система «Не трогай!»

Модули: ИК-датчик, пассивный зуммер, красный светодиод, Serial Monitor **Суть:** Если ИК-датчик засёк движение — включить сирену (зуммер пищит с переменной частотой через `tone()`) и мигать красным. В Serial вывести `"ТРЕВОГА!"`. Через 5 секунд без движения — система сбрасывается. **Цель:** Работа с `tone()`, `noTone()`, таймеры через `millis()`, состояния системы.

Задача 18. Автоматический шлагбаум

Модули: УЗ-датчик, серво-мотор, зелёный и красный светодиоды **Суть:** Если объект ближе 15 см — серво поднимает шлагбаум (90°), загорается зелёный. Если объект уходит — через 3 секунды шлагбаум опускается (0°), загорается красный. **Цель:** Конечный автомат (состояния: «закрыт» / «открыт» / «ожидание»), работа с `millis()` вместо `delay()`.

Задача 19. Метеостанция с логом

Модули: датчик температуры, датчик газа, модуль времени (RTC), Serial Monitor **Суть:** Каждые 10 секунд считывай температуру и уровень газа, получай время с RTC и выводи строку в Serial: `"12:05:30 | Темп: 24.5°C | Газ: 312 | Статус: НОРМА"`. Если газ выше порога — статус `"ОПАСНОСТЬ"`. **Цель:** Интеграция нескольких датчиков, форматированный протокол данных, пороговая логика.

Задача 20. Умный робот-охранник

Модули: УЗ-датчик, ИК-датчик, серво, 3 светодиода, зуммер, модуль времени, Serial Monitor **Суть:** Построй систему охраны помещения:

- Серво вращается как радар (0°–180°), на каждом шаге измеряет расстояние УЗ-датчиком.
- Если расстояние < 20 см — система переходит в режим «ТРЕВОГА»: зуммер, красный LED, серво останавливается и «смотрит» на объект.
- ИК-датчик работает как второй рубеж — при срабатывании усиливает тревогу (мигание всех LED).
- Время каждого события записывается в Serial с меткой RTC.
- Если 10 секунд нет обнаружений — система возвращается в режим патрулирования (зелёный LED).

Цель: Объединить все изученные модули в одну систему. Конечный автомат с несколькими состояниями, работа без `delay()`, структурированный код с функциями.

Таблица прогресса

№	Задача	Модули	Новый навык
1	Маяк	LED	`digitalWrite`, `delay`
2	SOS	LED	Цикл `for`
3	Светофор	3 LED	Много выходов
4	Бегущий огонь	3 LED	Последовательность
5	Диммер	LED (PWM)	`analogWrite`, ШИМ
6	Тревога газ	Газ + LED	`analogRead`, `if/else`
7	Охранник	ИК + LED	`digitalRead`
8	Линейка	УЗ + Serial	`pulseIn`, формулы
9	Парктроник	УЗ + 3 LED	`if/else if/else`
10	Термометр	Темп. + Serial	Математика с данными
11	Стрелка	Серво	`Servo.h`
12	Плавный поворот	Серво	`for` + серво
13	Радар	Серво + УЗ	Синхронизация устройств
14	Умный свет	Фоторезистор + LED	`map()`
15	Часы	RTC + Serial	Библиотека RTC
16	Будильник	RTC + зуммер + LED	Сравнение времени
17	Не трогай!	ИК + зуммер + LED	`tone()`, `millis()`
18	Шлагбаум	УЗ + серво + LED	Конечный автомат
19	Метеостанция	Темп. + газ + RTC	Мульти-датчики, лог
20	Робот-охранник	Все модули	Полная система